

RA4 – UNIDAD 4

Generación de Servicios en Red

4.1-EJERCICIO ALUMNOS

Desarrollar una REST API con Spring Boot que cargue datos desde un archivo JSON y permita realizar distintas peticiones GET utilizando ResponseEntity en todas las respuestas.

Los datos y formato iniciales que deberá de tener el archivo JSON son los siguientes:

```
[
  { "id": 1, "nombre": "Ane", "curso": "DAM1" },
  { "id": 2, "nombre": "Iker", "curso": "DAM2" },
  { "id": 3, "nombre": "Lucía", "curso": "DAM1" },
  { "id": 4, "nombre": "Markel", "curso": "DAM2" }
]
```

El archivo debe de estar localizado en el siguiente directorio:

src/main/resources/[alumnos.json]

Las peticiones a realizar tendrán que ser las siguientes:

1. *Obtener todos los alumnos (GET /alumnos)*: Devuelve el contenido completo del JSON.
2. *Buscar alumno por ID (GET /alumnos/{id})*:
 - Si existe: 200 OK.
 - Si no existe: 404 NOT FOUND.
3. *Buscar alumnos por curso (GET /alumnos/curso/{curso})*:
 - Si el curso existe: devuelve lista.
 - Si no hay resultados: 204 NO CONTENT.
4. *Devolver cantidad de alumnos (GET /alumnos/count)*: Debe responder lo siguiente { "total": 4 }.
5. *Buscar por nombre parcial (GET /alumnos/buscar?nombre=lu)*: Debe devolver todos los alumnos cuyo nombre contenga el texto buscado (sin distinguir mayúsculas / minúsculas).

El proyecto, tendrá que tener la siguiente estructura de paquetes / carpetas:

eus.tartanga.psp.alumnosjson/

./controller/

./service/

./model/

Conceptos a aplicar:

- Controladores REST.

- Servicios.
- Modelos.
- Lectura de ficheros JSON.
- Uso de ObjectMapper.
- Gestión de rutas y filtros.
- Buenas prácticas en arquitectura.

Requisitos técnicos

- Toda la lógica en el service.
 - Todas las respuestas con ResponseEntity.
 - El JSON debe cargarse usando ObjectMapper.
 - El controlador no puede leer el JSON directamente.
 - No modificar el nombre ni estructura del JSON.
-

4.2-EJERCICIO EMPLOYEES

Basándote en el proyecto proporcionado añade los endpoints necesarios para:

- Obtener el empleado cuyo ID se proporciona (si es null devolverá BadRequest y si no existe 404).
- Añadir un empleado (si es null devolverá BadRequest y si ya existe 409).
- Obtener el empleado cuyo firstName y lastName se proporcionan (si es null devolverá BadRequest y si no existe 404).
- Modificar un empleado pasando el ID y el nuevo objeto empleado (si no existe 404 y si va bien 200).
- Borrar un empleado pasando el ID (devuelve noContent si lo borra sin problemas y 404 si no existe).

Después, crea otro proyecto que trabaje contra un fichero JSON, en lugar de con ArrayList.

4.3- EJERCICIO WEBCLIENT ALUMNOS

Crear un cliente REST en Java que consuma la API de alumnos (/alumnos) utilizando WebClient de Spring y recibiendo los datos con bodyToMono. La clase de Alumnos debe de contener id, nombre y curso.

El proyecto Spring Boot tiene que tener la siguiente dependencia de WebFlux para poder usar WebClient:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Y no olvides también agregar:

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
```

```
<version>3.3.4</version>
</parent>
```

Las operaciones a realizar tanto en el main como en la clase que use WebClient tienen que ser las siguientes:

- Listar todos los alumnos. (GET /alumnos)
- Consultar un alumno por id. (GET /alumnos/{id})
- Consultar alumnos por curso. (GET /alumnos/curso/{curso})
- Consultar el número total de alumnos. (GET /alumnos/count)
- Crear un nuevo alumno. (POST /alumnos)
- Actualizar un alumno existente. (PUT /alumnos/{id})
- Borrar un alumno. (DELETE /alumnos/{id})

Opcional: Crear un menú en consola para que el usuario elija la operación a realizar.

Nota importante:

- Para recibir una lista de alumnos, utiliza `bodyToMono(Alumno[].class)` y conviértelo a `List<Alumno>` con `Arrays.asList()`.
- Para recibir un solo alumno, utiliza `bodyToMono(Alumno.class)`.
- Para operaciones de borrado, usa `bodyToMono(Void.class)`.
- Para simplificar la ejecución en consola, puedes usar `.block()` para obtener el resultado de manera síncrona.

Recomendaciones:

- Asegúrate de que el backend esté funcionando y tenga datos disponibles para consumir.
 - Usa `.block()` únicamente en ejercicios de consola; recuerda que WebClient es reactivo y puede usarse sin bloquear.
 - Coloca los recursos como `alumnos.json` en `src/main/resources` si el backend los lee desde archivos.
-